

Módulo 7

M7.1 Classes Abstratas e Interface

Um das premissas do POO é a abstração. Geralmente as linguagens de programação permitem o uso de classes abstratas para a definição de estrutura, de modo a padronizar a criação de outras classes, através da herança. Em POO alguns casos seria interessante descrever os campos e métodos que as classes herdeiras devem implementar, mas não permitir a criação de instâncias da classe ancestral. Dessa forma, a classe ancestral passaria a ser somente um guia de que métodos e campos deveriam ser implementados nas classes herdeiras. Em outras palavras, a classe ancestral daria para as classes descendentes o que deve ser feito, mas sem necessariamente dizer como deve ser feito. São mais comuns dois mecanismos que permitem a criação de classes que somente contêm descrições de campos e métodos que devem ser implementados, mas sem efetivamente implementar esses métodos. Classes que declaram mas não implementam métodos são particularmente úteis na criação de hierarquias de classes, porque não permitem a criação de instâncias delas e exigem que as classes descendentes implementem os métodos declarados nelas. Os dois mecanismos são classes *abstratas* e *interfaces*.

Classes Abstratas.

Um dos mecanismos de criação de superclasses com declarações mas sem a definição de todos os métodos permitindo porém a criação de métodos declarados como abstratos. Estes métodos abstratos são somente declarações como um método normal com seu nome, modificadores, tipo de retorno e lista de argumentos, mas sem tendo um corpo que contenha os comandos da linguagem que este método deva executar precedidos da palavra *abstract*. Se uma classe declara um método abstrato, as classes que herdarem desta deverão obrigatoriamente implementar o método abstrato com a mesma assinatura declaradas na classe ancestral.

Como visto, os métodos abstratos são declarados com o modificador *abstract*. Se uma classe tiver algum método abstrato, a classe também deverá obrigatoriamente ser declarada com o modificador *abstract*, mas podem haver classes abstratas sem nenhum método abstrato. Sempre reforçando que uma classe herdeira de uma classe abstrata é obrigada a implementar todos os métodos declarados como abstratos na classe ancestral,.

Métodos abstratos não podem ter corpo (a parte entre chaves). Somente a declaração do método é necessária, mas mesmo que variáveis passadas como argumentos nunca sejam usadas, seus nomes devem ser especificados.

Construtores de classes abstratas não podem ser abstratos. Mesmo que a classe abstrata não possa ser instanciada, seus construtores podem iniciar os campos da classe que serão usados por subclasses, sendo imprescindíveis em praticamente todos os casos.

Uma classe abstrata não pode ser instanciada diretamente, pois se fossem possíveis aconteceriam problemas pois faltariam as implementações dos métodos abstratos. Desta forma a instanciação de uma classe abstrata somente ocorre a partir das suas classes herdeiras.

Em UML uma classe abstrata é identificada com o nome da classe escrita em *itálico*.

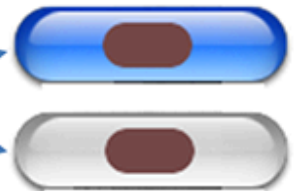


Exemplo:

Um botão dentro da programação tem vários atributos e métodos implementados mas esta classe exemplo tem como características a possibilidade de colocar um rótulo, e estar ativo ou desativo. Como é uma classe abstrata ela deixa a implementação da atividade para as suas classes herdeiras.

```
abstract class Botao
{
    protected string rotulo;
    private bool estado;

    public bool Estado
    {
        get { return estado; }
        set { estado = value; }
    }
    abstract public void Click();
    public void MostraBotao()
    {
        if (this.estado)
            Console.WriteLine("Botao {0} Ativo", this.rotulo);
        else
            Console.WriteLine("Botao {0} Inativo", this.rotulo);
    }
}
```



A classe abstrata Botão tem dois atributos, o atributo rótulo irá escrever no corpo do botão o texto definido pela classe herdeira. O segundo atributo define o estado do botão, se o botão estará ativo ou não. Note que o atributo estado é do tipo *bool*, ou seja, ele recebe os valores *true* ou *false*. Temos um método abstrato que fará com que seja obrigatório a implementação de uma ação no herdeiro quando o botão for acionado. Temos também um método concreto *MostraBotão()*, que mostra na tela o estado do botão. Este método sendo concreto não precisa ser implementado na classe herdeira, portanto ela faz parte do comportamento padrão que todo o botão irá ter.

```
class Ok : Botao
{
    public Ok()
    {
        base.rotulo = "Ok";
        Estado = true;
    }
    public override void Click()
    {
        if (Estado)
            Console.WriteLine("Calculando");
    }
}
```



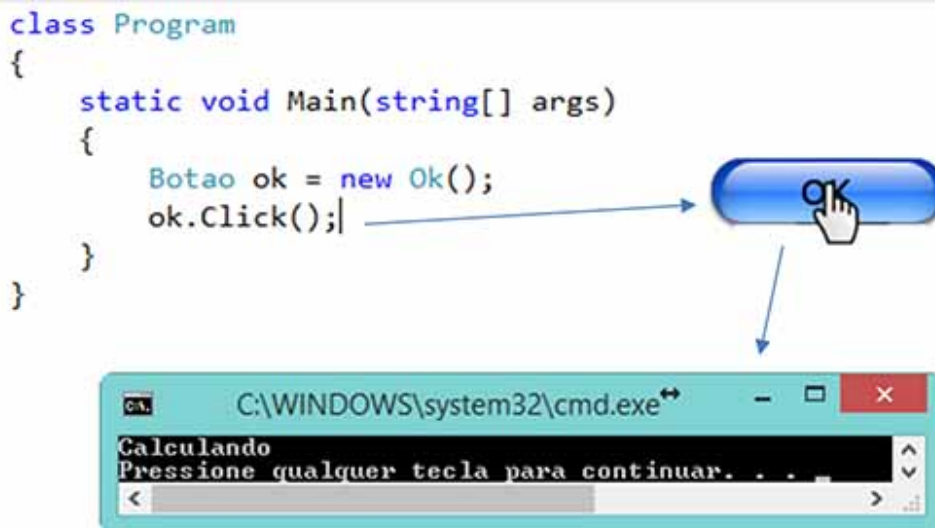
Como vimos vamos criar a classe de dois botões, a primeira será o botão Ok, O construtor irá definir o rótulo, e deixar o botão ativo. O método que irá atuar quando o botão for acionado. No nosso caso uma simulação de um cálculo, mostrando "calculando" na tela, mas na realidade pode ser um programa bem complexo. Este método não é facultativo pois a sua implementação está sendo imposta pelo método abstrato da superclasse.

A segunda classe herdeira será a do botão Cancelar. Este botão receberá o rótulo "Cancelar" e iniciará inativo. O seu método Click() irá simular as operações que vierem a ser feitos quando o botão for acionado, no nosso caso mostrando na tela "Cancelando". Note que em ambas as classes o botão somente irá funcionar quando o estado do botão estiver ativo (estado igual a *true*).

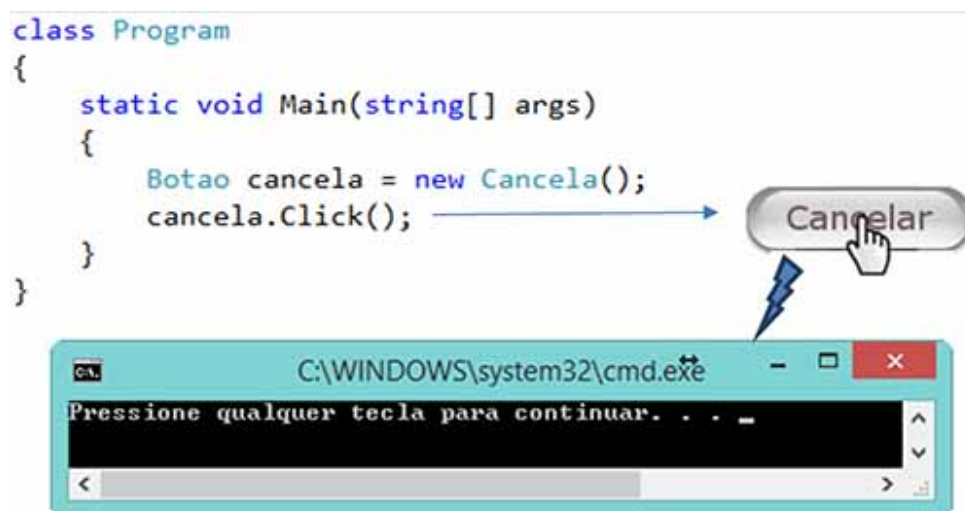
```
class Cancela : Botao
{
    public Cancela()
    {
        base.rotulo = "Cancelar";
        Estado = false;
    }
    public override void Click()
    {
        if (Estado)
            Console.WriteLine("Cancelando");
    }
}
```



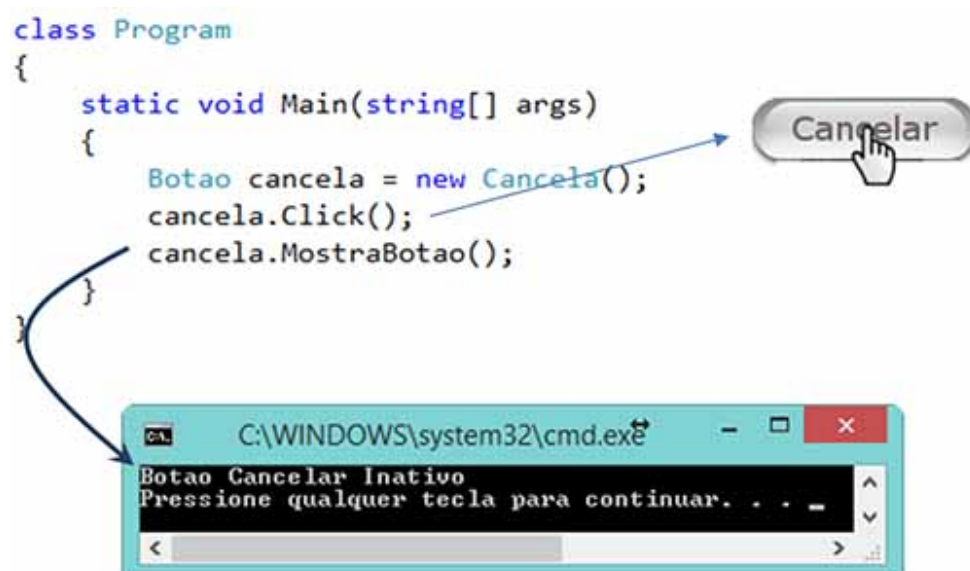
Para simular a execução do programa inicialmente vamos criar uma instância do botão Ok e acionar. Assim o método Click irá mostrar que o programa está calculando.



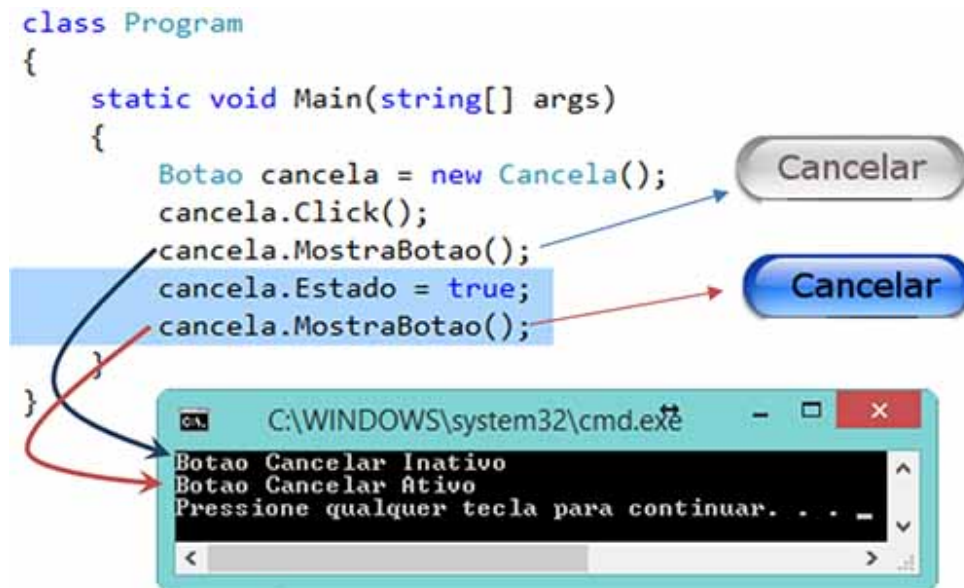
A segunda simulação é criando uma instância do botão cancela e acionando-a. Ao contrário do botão Ok, nada é mostrado na tela



Vamos verificar o estado do botão utilizando o método concreto MostraBotão() que está na superclasse.

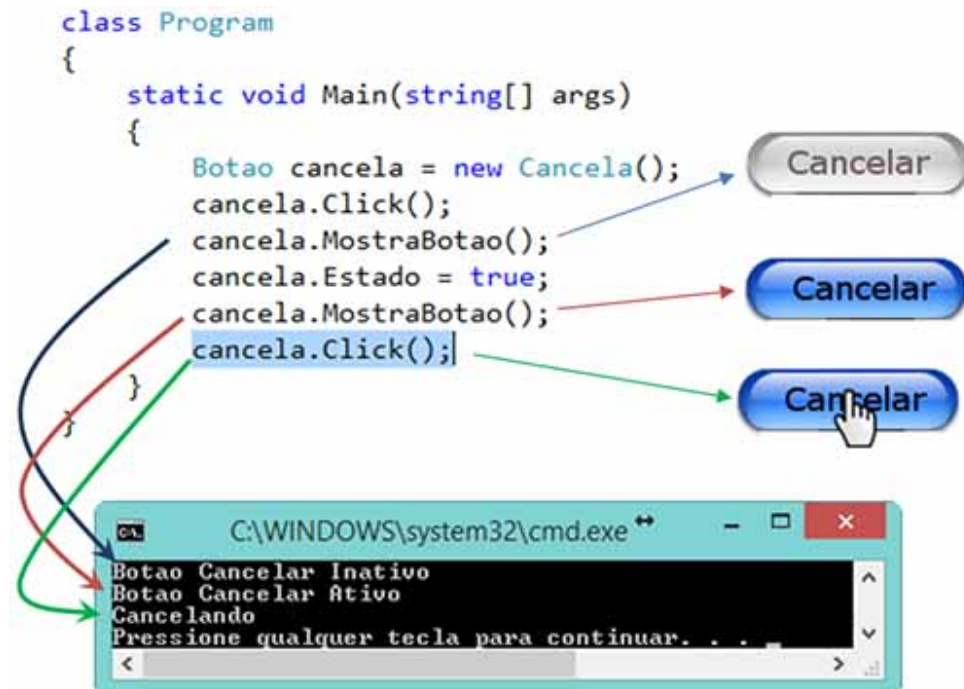


Para a classe que está utilizando a instância do botão Cancelar, o método `MostraBotão()` faz parte da classe `Cancela` pois ela é herdada da classe `Botão`. Assim ao utilizar o método vemos que o botão está desativado



Para ativar o botão Cancelar, mudamos o estado para `true`, novamente lembrando que o atributo estado não faz parte da classe `Cancela`, mas da superclasse `Botão`. Alterando o valor do atributo, novamente utilizamos o método `MostraBotao()` para verificar se o botão foi ativado

Assim o botão Cancelar fica ativo para podermos acionar, e na tela é mostrado que o método agora está trabalhando ("Cancelando").



• Interfaces

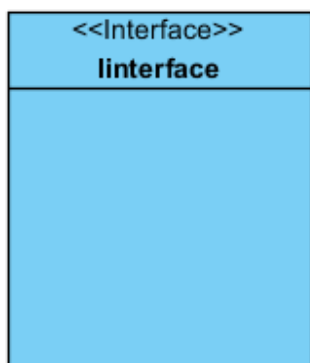
Podemos definir Interface como um contrato assinado entre duas ou mais partes.

Classes abstratas podem conter métodos não abstratos que serão herdados e poderão ser utilizados por instâncias de classes herdeiras. Se a classe não tiver nenhum método não abstrato, podemos criá-la como uma *interface*, que segue um modelo de declaração diferente do usado para classes mas tem funcionalidade similar à de classes abstratas.

Assim como uma classe abstrata, uma *interface* não pode ser instanciada. Todos os métodos na *interface* são implicitamente *abstract* e *public* e não podem ser declarados com seus corpos. Campos, se houverem, serão implicitamente considerados *static* e final devendo, portanto, ser iniciados na sua declaração.

Uma interface lista os serviços fornecidos por um componente. A interface é um contrato com o mundo exterior, que define exatamente o que uma entidade externa pode fazer com o objeto. Uma interface é o painel de controle do objeto (Sintes, 2002, p. 22).

Em UML a *interface* é representada com <<interface>> acima do nome da *interface*.



O prefixo dos nomes de *interface* é por convenção feita com a letra "I" para indicar que o tipo é uma *interface*. Assim para garantir que a definição de um par de classe/*interface* onde a classe é uma implementação padrão da *interface*, os nomes só diferem a letra que o prefixo no nome da *interface*.

Qualquer classe pode implementar uma ou várias *interfaces* simultaneamente. *Interfaces* desta forma podendo ser considerado um mecanismo simplificado de implementação de herança múltipla.

Muitos vezes o programador é confrontado com o complicado problema de decidir de qual classe herdar. Há muitas situações em que seria útil poder herdar de mais de uma classe. Por exemplo, imaginemos que temos uma classe-base que representa leitores de CD, assim como uma outra classe-base que representa leitores de cassetes. De qual classe-base deveria herdar uma classe que represente uma aparelhagem de som ?

Talvez, neste caso fizesse mais sentido utilizar composição (juntar a classe cassete e cd em uma só). No entanto, ao utilizar composição perdemos a capacidade de

converter um objeto numa classe-base e utilizá-lo independentemente das suas especificidades.

Para resolver este tipo de problemas, em C# existe o conceito de interface. Uma interface permite obter quase a totalidade dos benefícios da herança múltipla, mas sem os seus problemas. Uma interface especifica um conjunto de métodos que tem que ser implementados por uma classe. Uma classe pode implementar uma ou mais interfaces, bastando para isso indicar quais as interfaces implementadas, tendo os respectivos métodos definidos. As interfaces são extremamente importantes e amplamente utilizadas tanto na plataforma .Net como na programação do dia-a-dia."

Definição do livro C# 2.0

Outra parte : interface é como um contrato onde pelo menos 2 elementos o assinam ?

O .Net permite que seja criada uma interface e que esta interface seja assinada por apenas um único objeto.

Pensando no sentido de Contrato, como as literaturas explicam, um contrato nunca é assinado sozinho. Por exemplo, se você fizer o contrato de compra e venda de uma casa, tanto o vendedor como o comprador assinam o mesmo contrato. Logo, duas assinaturas.

Isso nos leva a sua dúvida. Se duas pessoas assinam um contrato, faria sentido, de forma análoga, que apenas um objeto assinasse a interface ?

Difícil dizer que faz sentido. Mas não podemos afirmar que está tecnicamente errado, tendo em vista que esta implementação é aceita pelo compilador.

Porém, um argumento que talvez faça sentido seria o seguinte :

Pense em um objeto que seja responsável pela autenticação de um usuário. Normalmente, em uma aplicação orientada a objetos, esse objeto é único. Vamos supor que, para atender as regras de negócios da sua empresa ou a padrões de segurança, para que um usuário seja autenticado, você deva seguir um conjunto obrigatório de procedimentos.

A utilização da Interface, neste caso, seria uma forma de garantir que o objeto responsável pela autenticação implemente os passos estipulados. Apesar de que, o fato da implementação do método pode não ter "recheio" (um método vazio) e isso não garantiria que o passo seria executado.

Por outro lado, tornaria a implementação formalizada e, arquiteturalmente falando, tornaria a leitura do código bastante clara.

Desta forma, faria sentido sim ter uma interface implementada por um único objeto...

Por fim, falando das classes abstratas, podemos dizer que ela traz consigo as características de uma classe base aliada as características de CONTRATO como acontece com a interface. Porém, como você já herdou uma vez, não poderá ter nova herança na classe-filha.

Nesse momento você pode fazer uma opção. Ao invés de utilizar a classe abstrata, você pode ter na classe-base apenas as operações e propriedades que você tem certeza que serão genéricos e, para operações que você deseja garantir que existam, mas não tem como implementar já na classe-base a operação propriamente dita, você

pode utilizar a Interface para fazer o papel do CONTRATO, obrigando a classe-filha a ter o método em questão.

Um outro uso interessante para as *Interfaces* é para implementar bibliotecas de constantes, pois os campos declarados em uma *interface* devem ser declarados como estáticos e inicializados, desta forma pode-se ter *interfaces* somente com campos e sem métodos assim qualquer classe que a venha a implementar terá disponível os seus atributos. Assim a técnica que possibilita uma maior padronização é a *interface*. A *interface* padroniza o formato da classe, ou seja, como deverão ser suas entradas e saídas de dados, deixando a implementação dos processos a cargo de cada classe que herdou essas *interfaces*.

Uma interface apenas declara apenas os métodos que obrigarão as suas classes a implementar, desta forma ela só tem declarações de métodos, sem possuir atributos e construtores. Os métodos declarados em uma *interface* podem ser sobrecarregados, ou seja, podem existir métodos com nomes diferentes porém com assinaturas diferentes.

Em C# ela é implementada utilizando a palavra *interface* no lugar de *class*. No corpo são feitas as declarações dos métodos que serão padronizados pela *interface*. Na classe que implementa a *interface* são definidos os métodos impostos pela *interface*.

```
interface IexemploInterface
{
    void MetodoExemplo();
    void MetodoExemplo1();
}

class ClasseImplementadora : IexemploInterface
{
    void IexemploInterface.MetodoExemplo()
    {
        // Implementação do método.
    }
    void IexemploInterface.MetodoExemplo1()
    {
        // Implementação do método.
    }
}
```

O processo de criação de uma *interface*, normalmente se cria uma referência do tipo da *interface*, e a instância é feita utilizando a classe que implementa a *interface*

```
class program
{
    static void Main()
    {
        IexemploInterface obj = new ClasseImplementadora ();
        obj.MetodoExemplo();
    }
}
```

M7.2 Herança simples e múltipla

Herança é utilizada para se representar a similaridade entre diferentes classes, com isso se simplifica a definição de Classes semelhantes a outras previamente definidas.

Com isso é possível que membros comuns, métodos e atributos sejam definidos somente uma vez (encapsulamento), e ainda especializar estes membros em determinados casos.

Além do encapsulamento de dados, herança traz outros benefícios, tais como: permite a reutilização de software em que novas classes são criadas a partir de classes pré-existent, absorvendo seus atributos e comportamentos adicionando novos recursos as classes antigas.

A reutilização de software apresenta algumas vantagens, por exemplo, economiza e facilita no desenvolvimento de novos programas, incentiva a reutilização de códigos de alta qualidade já testados e depurados, reduzindo possíveis problemas depois da instalação software final.

A herança define uma relação entre classes do tipo "é - um", onde uma classe compartilha a estrutura e o comportamento definidos em uma ou mais classes. A partir do reconhecimento da similaridade entre as classes definem uma hierarquia de classes, subdividida em:

Superclasses (classe base ou classe mãe): Representam abstrações mais gerais e se subdividem em duas:

- Direta: A subclasse herda explicitamente da superclasse.

- Indireta : É herdada de dois ou mais níveis acima na hierarquia de classes.

Subclasses (classes derivadas ou classes filha): Representam abstrações em que os atributos e os serviços específicos são adicionados, modificados e/ou removidos.

Exemplo :

Imagine que em um determinado sistema orientado a objetos exista uma classe chamada Cachorro. Ao definirmos a classe Cachorro, encontramos como atributos (propriedades ou características), Cor, Raça e Idade, além das operações Latir e Beber Leite. Imagine agora que será necessário criar a classe Gato. Ao definirmos seus atributos, percebemos que a classe Gato teria Cor, Raça e Idade, da mesma forma que a classe Cachorro, acrescido apenas da operação Miar.

Ao invés de termos duas classes parecidos, podemos aplicar o conceito de herança da seguinte forma :

Criamos a classe base Animal Terrestre, com as propriedades Cor,Raça e Idade e com a operação Beber Leite. A partir desta classe base, criamos outras duas subclasses –

Cachorro e Gato, que herdam na classe base e especializamos apenas as operações que as diferem, Latir para a classe Cachorro e Miar para a classe Gato.

Desta forma teríamos uma herança simples.

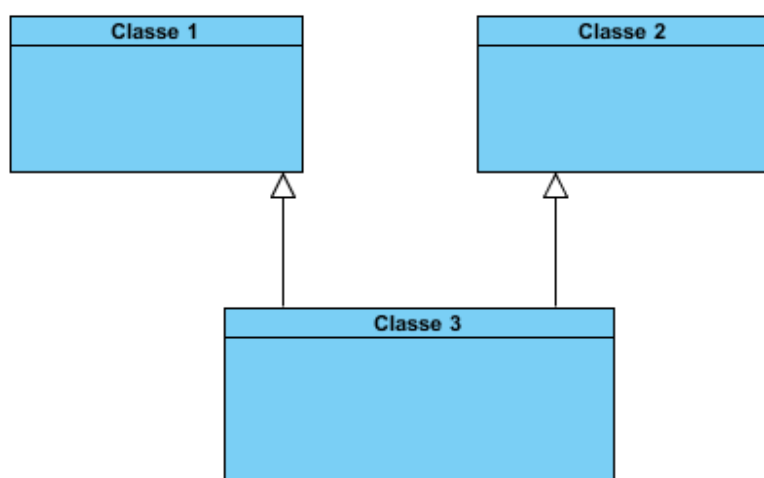
Quando uma classe é uma derivação de uma classe já existente, temos uma relação de herança. Neste caso uma herança simples.

Existe um segundo conceito chamado herança múltipla, que significa que uma classe pode ser derivada de mais de uma classe já existente.

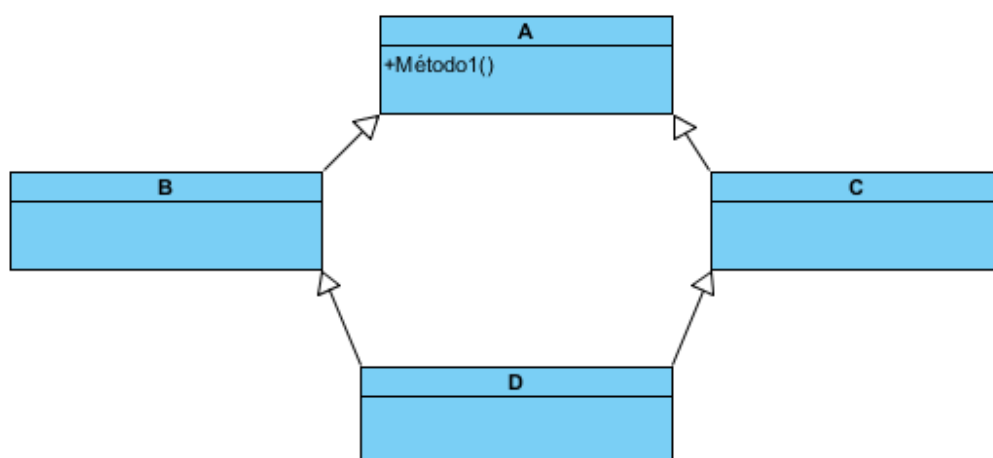
A princípio a herança múltipla traz certas vantagens, pois teríamos uma classe derivada de diversas classes já existentes. Porém o fato é que o controle desse tipo de estrutura é bem complexo e são poucas linguagens de programação permitem realizar este conceito. Normalmente as linguagens de programação permitem realizar apenas a herança simples.

Por outro lado, a modelagens visando herança múltipla pode trazer esta complexidade para a manutenção do sistema, gerando assim uma maior mão de obra se tratando de atualizações e alterações no sistema.

Como não são todas as linguagens de programação que permitem herança múltipla (uma classe herdar características de mais de uma classe), a técnica de *interface* nos permite agregar em uma classe várias *interfaces* distintas.



A herança múltipla pode causar um sério problema, o chamado diamante da morte. Ela acontece quando uma herança múltipla de duas classes que são herdeiras de uma mesma classe (Figura 115). As classes B e C herdarão o método1 da classe A. A classe D ao herdar de B e de C receberá o método1 de ambos causando o problema já que teremos uma duplicidade, ocorrendo aí um erro. Algumas linguagens estabelecem uma hierarquia para contornar o problema.



A linguagem C#, assim como o Java não permite o uso de herança múltipla. A situação pode ser simulada com o uso de *interface*, mas ficando claro que não é uma herança múltipla dentro da sua definição. Como um exemplo simples, é uma classe que recebe duas *interfaces* uma que determina características de um meio de

transporte voador, e outra que determina as características de um meio de transporte navegador. No caso um avião é um voador (Código 87), um transatlântico (que não está no exemplo) um navegador, e o caso de uma múltipla herança, um hidroavião.

```
public interface IVoador
{
    void Voar();
}

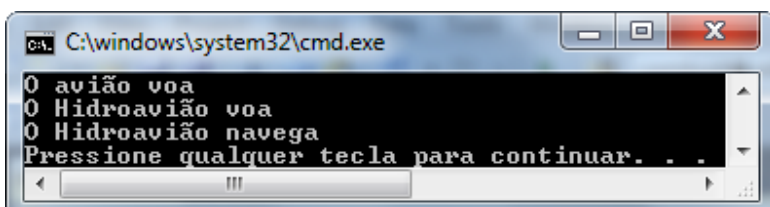
public interface INavegador
{
    void Navegar();
}

public class Aviao : IVoador
{
    public void Voar()
    {
        Console.WriteLine("O avião voa");
    }
}

public class HidroAviao : IVoador, INavegador
{
    public void Voar()
    {
        Console.WriteLine("O Hidroavião voa");
    }
    public void Navegar()
    {
        Console.WriteLine("O Hidroavião navega");
    }
}
```

Uma vez montadas as classes vamos montar a classe que irá testar as classes. Ela irá criar uma instância de um avião chamado "boeing" que voará, e de um hidroavião chamado "catalina" que voará e navegará .

```
class Program
{
    static void Main(string[] args)
    {
        IVoador boeing = new Aviao();
        boeing.Voar();
        HidroAviao catalina = new HidroAviao();
        catalina.Voar();
        catalina.Navegar();
    }
}
```



Exercício 1:

Podemos definir Interface como:

A)

um contrato assinado entre duas partes.

B)

um contrato assinado entre três ou mais partes.

C)

um contrato assinado por apenas uma classe.

D)

um contrato assinado entre duas ou mais partes.

E)

Uma classe intermediária entre a classe herdeira e a superclasse

Exercício 2:

B2-3 2

Dado o código abaixo

```

1  class Data
2  {
3      private int dia, mes, ano;
4      public Data(int d, int m, int a)
5      {
6          this.dia = d;
7          this.mes = m;
8          this.ano = a;
9      }
10     public void setDia(int d)
11     {
12         this.dia = d;
13     }
14     public void setMes(int m)
15     {
16         this.mes = m;
17     }
18     public void setAno(int a)
19     {
20         this.ano = a;
21     }
22     public override string ToString()
23     {
24         return Convert.ToString(this.dia)+"/"+Convert.ToString(this.mes)+"/"+Convert.ToString(this.ano);
25     }
26 }

```

```

27 class Pessoa
28 {
29     private String nome;
30     private Data nascimento;
31     public Pessoa(String n, Data d)
32     {
33         this.nome = n;
34         this.nascimento = d;
35     }
36     public void setNome(String n)
37     {
38         this.nome = n;
39     }
40     public String getNome()
41     {
42         return this.nome;
43     }
44     public void setNascimento(Data d)
45     {
46         this.nascimento = d;
47     }
48     public Data getNascimento()
49     {
50         return this.nascimento;
51     }
52 }

```

```

54 class Empresa
55 {
56     private Pessoa empregado;
57     private Data contratacao;
58     public Empresa(Pessoa p, Data c)
59     {
60         this.empregado = p;
61         this.contratacao = c;
62     }
63     public void setEmpregado(Pessoa p)
64     {
65         this.empregado = p;
66     }
67     public Pessoa getEmpregado()
68     {
69         return this.empregado;
70     }
71     public void setContratacao(Data d)
72     {
73         this.contratacao = d;
74     }
75     public Data getContratacao()
76     {
77         return this.contratacao;
78     }
79 }

```

```

81 class Program
82 {
83     static void Main(string[] args)
84     {
85         Data n = new Data(11, 9, 2011);
86         Pessoa p = new Pessoa("José", n);
87         Data c = new Data(1, 4, 2010);
88         Empresa e = new Empresa(p, c);
89         // ----- instrução -----
90     }
91 }
92
93
94 }

```

Qual a instrução a ser colocada na linha 90 para alterar o ano do nascimento da pessoa p para 2001?

A)

```
e.getEmpregado(getNascimento(setAno(2001)));
```

B)

```
e.setEmpregado(setNascimento(setAno(2001)));
```

C)

```
e.pessoa.nascimento.ano=2001;
```

D)

```
e.empregado.nascimento.ano=2001;
```

E)

```
e.getEmpregado().getNascimento().setAno(2001);
```

Exercício 3:

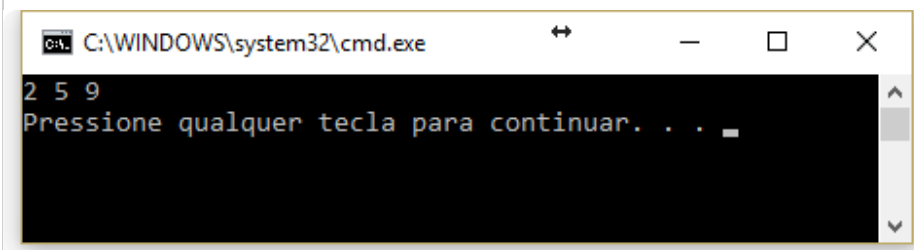
B2-3 4

Considere o código abaixo:

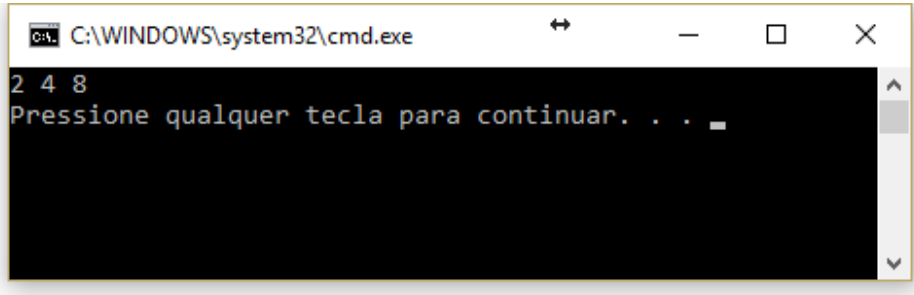
```
1      abstract class Classe1
2      {
3          public virtual int soma(int a)
4          {
5              return a + 1;
6          }
7      }
8
9      class Classe2 : Classe1
10     {
11         public override int soma(int a)
12         {
13             return a + 2;
14         }
15     }
16     class Classe3 : Classe2
17     {
18         public int soma(int a)
19         {
20             return base.soma(a + 3);
21         }
22     }
23     class Classe4 : Classe3
24     {
25         public int soma(int a)
26         {
27             return base.soma(a + 4);
28         }
29     }
30     class Program
31     {
32         static void Main(string[] args)
33         {
34             Classe2 c2 = new Classe2();
35             Classe3 c3 = new Classe3();
36             Classe4 c4 = new Classe4();
37             Console.WriteLine("{0} {1} {2}", c2.soma(0), c3.soma(0), c4.soma(0));
38         }
39     }
40
```

Qual a saída ao executar o programa?

A)

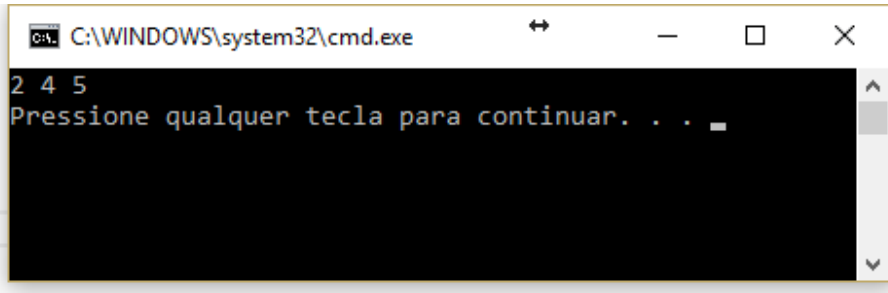


B)



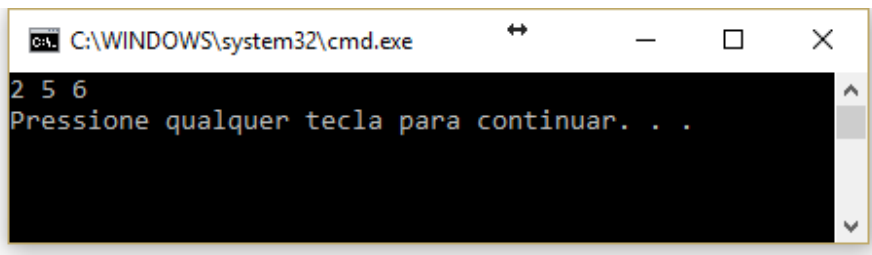
```
C:\WINDOWS\system32\cmd.exe
2 4 8
Pressione qualquer tecla para continuar. . . .
```

C)



```
C:\WINDOWS\system32\cmd.exe
2 4 5
Pressione qualquer tecla para continuar. . . .
```

D)



```
C:\WINDOWS\system32\cmd.exe
2 5 6
Pressione qualquer tecla para continuar. . . .
```

E)

Erro,,o programa não compila

Exercício 4:

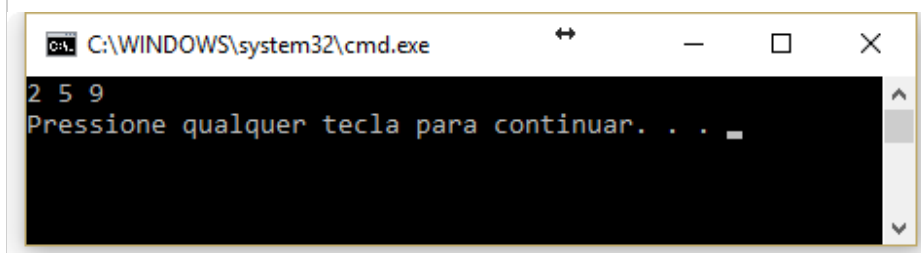
B2-3 5

Considere o código abaixo:

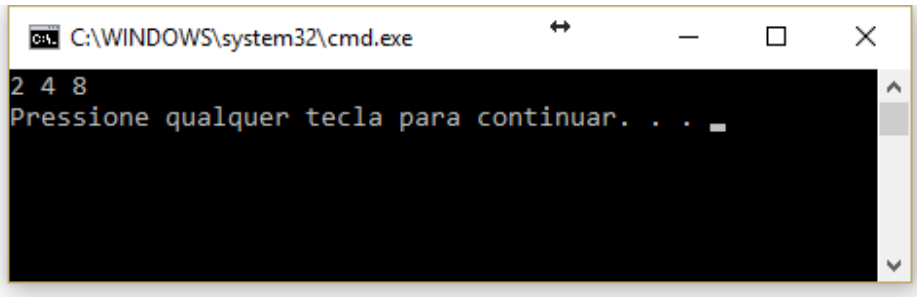
```
1  abstract class Classe1
2  {
3      public virtual int soma(int a)
4      {
5          return a + 1;
6      }
7
8  }
9  class Classe2 : Classe1
10 {
11     public override int soma(int a)
12     {
13         return a + 2;
14     }
15 }
16 class Classe3 : Classe1
17 {
18     public int soma(int a)
19     {
20         return base.soma(a + 3);
21     }
22 }
23 class Classe4 : Classe1
24 {
25     public int soma(int a)
26     {
27         return base.soma(a + 4);
28     }
29 }
30 class Program
31 {
32     static void Main(string[] args)
33     {
34         Classe2 c2 = new Classe2();
35         Classe3 c3 = new Classe3();
36         Classe4 c4 = new Classe4();
37         Console.WriteLine("{0} {1} {2}", c2.soma(0), c3.soma(0), c4.soma(0));
38     }
39 }
40
```

Qual a saída ao executar o programa?

A)

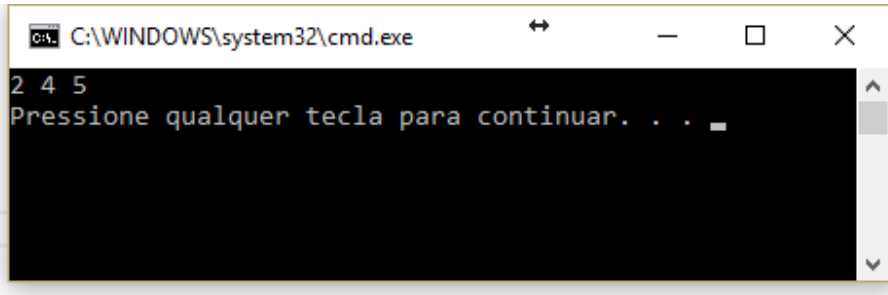


B)



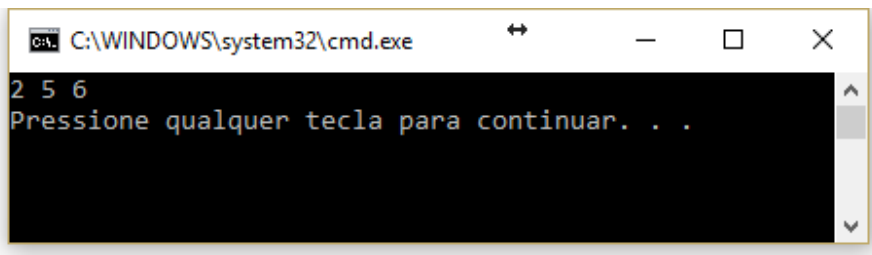
```
C:\WINDOWS\system32\cmd.exe
2 4 8
Pressione qualquer tecla para continuar. . . .
```

C)



```
C:\WINDOWS\system32\cmd.exe
2 4 5
Pressione qualquer tecla para continuar. . . .
```

D)



```
C:\WINDOWS\system32\cmd.exe
2 5 6
Pressione qualquer tecla para continuar. . . .
```

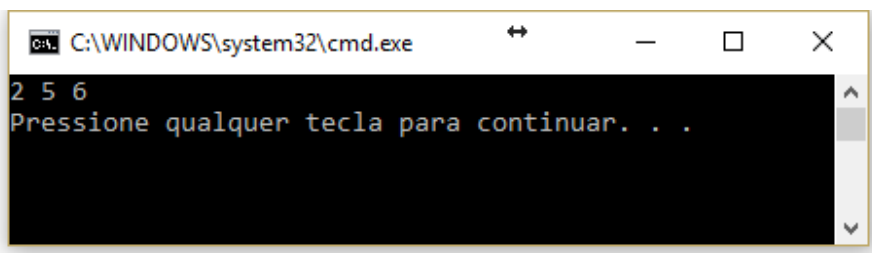
E)

Erro, o Programa não compila.

Exercício 5:

B2-3 6

Dada a saída:



```
C:\WINDOWS\system32\cmd.exe
2 5 6
Pressione qualquer tecla para continuar. . . .
```

Qual dos programas abaixo a gera quando executado?

A)

```
1  abstract class Classe1
2  {
3      public virtual int soma(int a)
4      {
5          return a + 1;
6      }
7
8  }
9  class Classe2 : Classe1
10 {
11     public override int soma(int a)
12     {
13         return a + 2;
14     }
15 }
16 class Classe3 : Classe2
17 {
18     public int soma(int a)
19     {
20         return base.soma(a + 3);
21     }
22 }
23 class Classe4 : Classe3
24 {
25     public int soma(int a)
26     {
27         return base.soma(a + 4);
28     }
29 }
30 class Program
31 {
32     static void Main(string[] args)
33     {
34         Classe2 c2 = new Classe2();
35         Classe3 c3 = new Classe3();
36         Classe4 c4 = new Classe4();
37         Console.WriteLine("{0} {1} {2}", c2.soma(0), c3.soma(0), c4.soma(0));
38     }
39 }
40
```

B)

```
1      abstract class Classe1
2      {
3          public virtual int soma(int a)
4          {
5              return a + 1;
6          }
7
8      }
9      class Classe2 : Classe1
10     {
11         public override int soma(int a)
12         {
13             return a + 2;
14         }
15     }
16     class Classe3 : Classe1
17     {
18         public int soma(int a)
19         {
20             return base.soma(a + 3);
21         }
22     }
23     class Classe4 : Classe3
24     {
25         public int soma(int a)
26         {
27             return base.soma(a + 4);
28         }
29     }
30     class Program
31     {
32         static void Main(string[] args)
33         {
34             Classe2 c2 = new Classe2();
35             Classe3 c3 = new Classe3();
36             Classe4 c4 = new Classe4();
37             Console.WriteLine("{0} {1} {2}", c2.soma(0), c3.soma(0), c4.soma(0));
38         }
39     }
40
```

C)

```
1  abstract class Classe1
2  {
3      public virtual int soma(int a)
4      {
5          return a + 1;
6      }
7
8  }
9  class Classe2 : Classe1
10 {
11     public override int soma(int a)
12     {
13         return a + 2;
14     }
15 }
16 class Classe3 : Classe1
17 {
18     public int soma(int a)
19     {
20         return base.soma(a + 3);
21     }
22 }
23 class Classe4 : Classe1
24 {
25     public int soma(int a)
26     {
27         return base.soma(a + 4);
28     }
29 }
30 class Program
31 {
32     static void Main(string[] args)
33     {
34         Classe2 c2 = new Classe2();
35         Classe3 c3 = new Classe3();
36         Classe4 c4 = new Classe4();
37         Console.WriteLine("{0} {1} {2}", c2.soma(0), c3.soma(0), c4.soma(0));
38     }
39 }
40
```

D)

```
1  abstract class Classe1
2  {
3      public virtual int soma(int a)
4      {
5          return a + 1;
6      }
7
8  }
9  class Classe2 : Classe1
10 {
11     public override int soma(int a)
12     {
13         return a + 2;
14     }
15 }
16 class Classe3 : Classe2
17 {
18     public int soma(int a)
19     {
20         return base.soma(a + 3);
21     }
22 }
23 class Classe4 : Classe2
24 {
25     public int soma(int a)
26     {
27         return base.soma(a + 4);
28     }
29 }
30 class Program
31 {
32     static void Main(string[] args)
33     {
34         Classe2 c2 = new Classe2();
35         Classe3 c3 = new Classe3();
36         Classe4 c4 = new Classe4();
37         Console.WriteLine("{0} {1} {2}", c2.soma(0), c3.soma(0), c4.soma(0));
38     }
39 }
40
```

E)

```
1  class Classe1: Classe2
2  {
3      public Classe1()
4      {
5          Console.WriteLine("a");
6      }
7      public Classe1(String st) :base(st)
8      {
9          Console.WriteLine(st+"\nb");
10     }
11 }
12 class Classe2
13 {
14     public Classe2()
15     {
16         Console.WriteLine("c");
17     }
18     public Classe2(String st)
19     {
20         Console.WriteLine(st + "\nd");
21     }
22 }
23 class Program
24 {
25     static void Main(string[] args)
26     {
27         Classe1 c1 = new Classe1("e");
28         Classe2 c2 = new Classe2();
29     }
30 }
```

Exercício 6:

Considere o código abaixo:


```
1      class Classe1
2      {
3          public Classe1()
4          {
5              Console.WriteLine("Lugar1");
6          }
7      }
8      class Classe2 : Classe1
9      {
10         public Classe2()
11         {
12             Console.WriteLine("Lugar2");
13         }
14     }
15     class Program
16     {
17         static void Main(string[] args)
18         {
19             Classe1 c1 = new Classe1();
20             Classe2 c2 = new Classe2();
21         }
22     }
```

Qual o resultado mostrado na tela e qual a sequência de saída? Qual o resultado mostrado na tela e qual a sequência de saída?

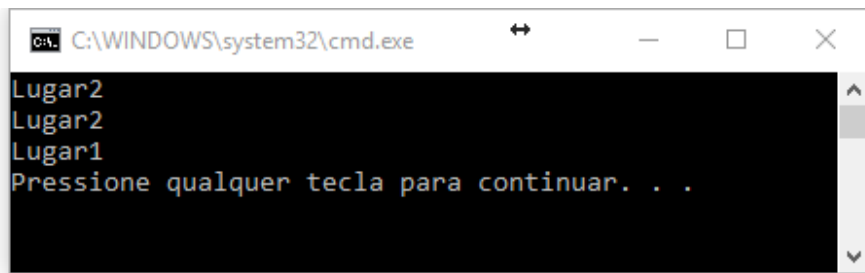
A)

19, c1-1, c1-3, c1-5, 20, c2-8, c2-1, c2-1, c2-3, c2-5, c2-8, c2-10, c2-12

B)

19, c1-8, c1-1, c1-1, c1-3, c1-5, c1-8, c1-10, c1-12,20, c2-1, c2-3, c2-5

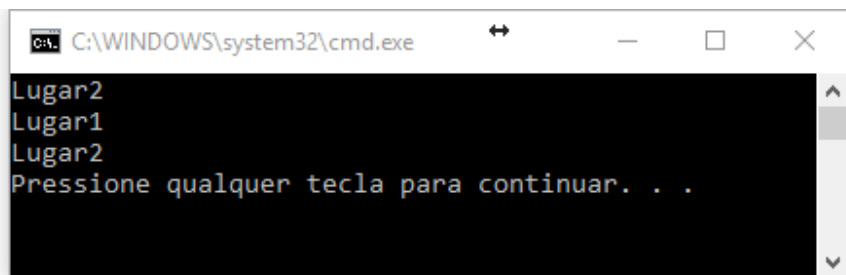
C)



```
C:\WINDOWS\system32\cmd.exe
Lugar2
Lugar2
Lugar1
Pressione qualquer tecla para continuar. . .
```

19, c1-8, c1-10, c1-12, 20, c2-1, c1-3, c2-5, c2-8, c2-10, c2-12

D)



```
C:\WINDOWS\system32\cmd.exe
Lugar2
Lugar1
Lugar2
Pressione qualquer tecla para continuar. . .
```

19, c1-1, c1-8, c1-10, c1-12, c1-1, c1-3, c1-5, 20, c2-8, c2-10, c2-12

E)

Erro, não compila

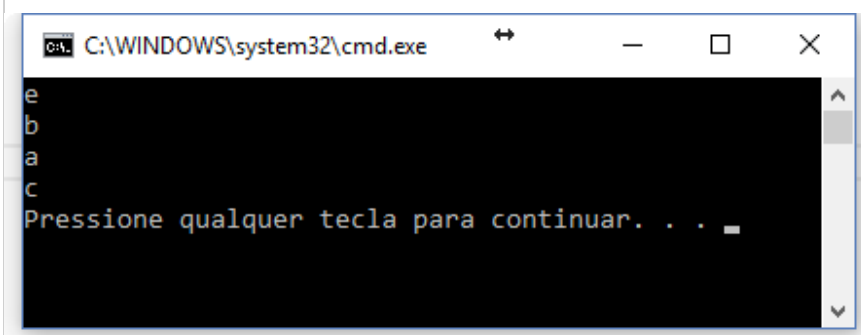
Exercício 7:

Considere o código abaixo:

```
1  class Classe1
2  {
3      public Classe1()
4      {
5          Console.WriteLine("a");
6      }
7      public Classe1(String st)
8      {
9          Console.WriteLine(st+"\nb");
10     }
11 }
12 class Classe2 : Classe1
13 {
14     public Classe2()
15     {
16         Console.WriteLine("c");
17     }
18     public Classe2(String st):base(st)
19     {
20         Console.WriteLine(st + "\nd");
21     }
22 }
23 class Program
24 {
25     static void Main(string[] args)
26     {
27         Classe1 c1 = new Classe1("e");
28         Classe2 c2 = new Classe2();
29     }
30 }
```

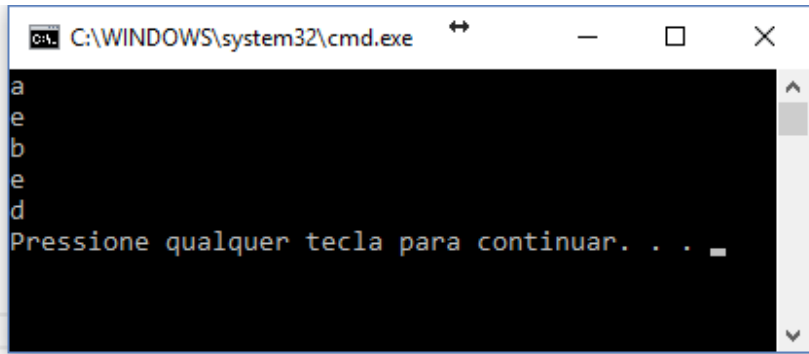
Qual a alternativa que mostra a tela, e a sequência de execução correta.

A)



27, c1-1, c1-7, c1-9, 28, c2-12, c2-1, c2-3, c2-5, c2-12, c2-14, c2-16

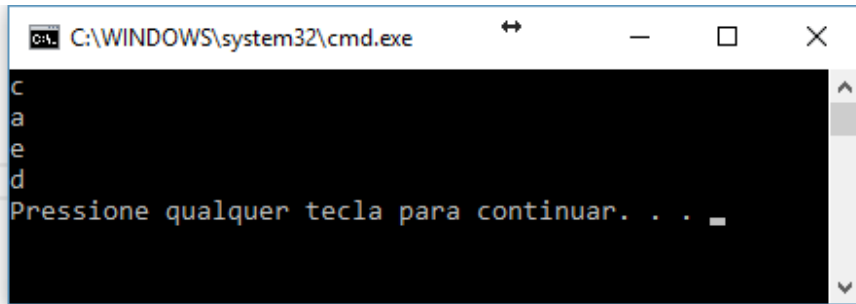
B)



```
C:\WINDOWS\system32\cmd.exe
a
e
b
e
d
Pressione qualquer tecla para continuar. . .
```

27-c1-1, c1-3, c1-5, 28, c2-12, c2-18, c2-1, c1-7, c2-9, c2-20

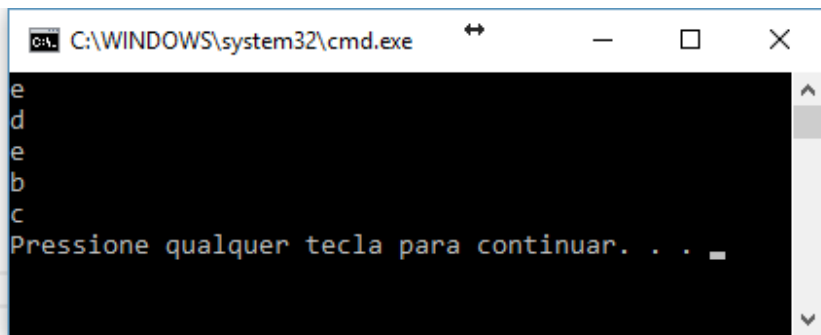
C)



```
C:\WINDOWS\system32\cmd.exe
c
a
e
d
Pressione qualquer tecla para continuar. . .
```

28, c1-1, c1-12, c1-14, c1-16, c1-1, c1-3, c1-5, 28, c2-12, c2-18, c2-20

D)



```
C:\WINDOWS\system32\cmd.exe
e
d
e
b
c
Pressione qualquer tecla para continuar. . .
```

28, c1-1, c1-7, c1-18, c1-20, c1-7-c1-9, 28, c1-12, c1-14, c1-16

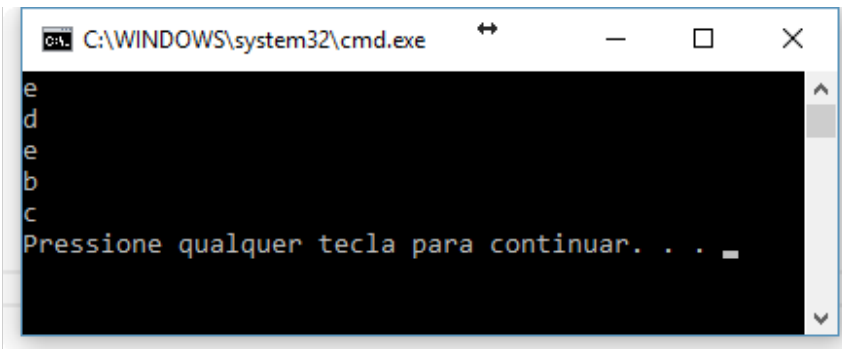
E)

Erro, Não Compiila

Exercício 8:

B2-4 4

Dada a tela abaixo:



Qual o código que gerou a tela?

A)

```
1      class Classe1
2      {
3          public Classe1()
4          {
5              Console.WriteLine("a");
6          }
7          public Classe1(String st)
8          {
9              Console.WriteLine(st+"\nb");
10         }
11     }
12     class Classe2 : Classe1
13     {
14         public Classe2()
15         {
16             Console.WriteLine("c");
17         }
18         public Classe2(String st):base(st)
19         {
20             Console.WriteLine(st + "\nd");
21         }
22     }
23     class Program
24     {
25         static void Main(string[] args)
26         {
27             Classe1 c1 = new Classe1("e");
28             Classe2 c2 = new Classe2();
29         }
30     }
```

B)

```
1  class Classe1
2  {
3      public Classe1()
4      {
5          Console.WriteLine("a");
6      }
7      public Classe1(String st)
8      {
9          Console.WriteLine(st+"\nb");
10     }
11 }
12 class Classe2 : Classe1
13 {
14     public Classe2()
15     {
16         Console.WriteLine("c");
17     }
18     public Classe2(String st):base(st)
19     {
20         Console.WriteLine(st + "\nd");
21     }
22 }
23 class Program
24 {
25     static void Main(string[] args)
26     {
27         Classe1 c1 = new Classe1();
28         Classe2 c2 = new Classe2("e");
29     }
30 }
```

C)

```
1      class Classe1: Classe2
2      {
3          public Classe1()
4          {
5              Console.WriteLine("a");
6          }
7          public Classe1(String st) :base(st)
8          {
9              Console.WriteLine(st+"\nb");
10         }
11     }
12     class Classe2
13     {
14         public Classe2()
15         {
16             Console.WriteLine("c");
17         }
18         public Classe2(String st)
19         {
20             Console.WriteLine(st + "\nd");
21         }
22     }
23     class Program
24     {
25         static void Main(string[] args)
26         {
27             Classe1 c1 = new Classe1();
28             Classe2 c2 = new Classe2("e");
29         }
30     }
```

D)

```
1  class Classe1: Classe2
2  {
3      public Classe1()
4      {
5          Console.WriteLine("a");
6      }
7      public Classe1(String st) :base(st)
8      {
9          Console.WriteLine(st+"\nb");
10     }
11 }
12 class Classe2
13 {
14     public Classe2()
15     {
16         Console.WriteLine("c");
17     }
18     public Classe2(String st)
19     {
20         Console.WriteLine(st + "\nd");
21     }
22 }
23 class Program
24 {
25     static void Main(string[] args)
26     {
27         Classe1 c1 = new Classe1("e");
28         Classe2 c2 = new Classe2();
29     }
30 }
```

E)


```
1  class Classe1: Classe2
2  {
3      public Classe1()
4      {
5          Console.WriteLine("a");
6      }
7      public Classe1(String st)
8      {
9          Console.WriteLine(st+"\nb");
10     }
11 }
12 class Classe2
13 {
14     public Classe2()
15     {
16         Console.WriteLine("c");
17     }
18     public Classe2(String st):base(st)
19     {
20         Console.WriteLine(st + "\nd");
21     }
22 }
23 class Program
24 {
25     static void Main(string[] args)
26     {
27         Classe1 c1 = new Classe1("e");
28         Classe2 c2 = new Classe2();
29     }
30 }
```